

CRC Package Interface Specification

Current API design

Table of contents

1 Overview	1
2 Main arguments	2
3 Current workflow	2
4 Parser responsibilities	3
4.1 <code>parse_captures()</code>	3
4.2 <code>parse_outcome()</code>	3
4.3 <code>parse_capture_formula()</code>	3
4.4 <code>parse_outcome_formula()</code>	4
4.5 <code>parse_misclass()</code>	4
5 Returned object	4
6 Next implementation steps	5

1 Overview

The package is centred around a single high-level function:

```
crc_fit(  
  data,  
  captures,  
  capture_formula,  
  outcome = NULL,  
  outcome_formula = NULL,  
  outcome_dist = c("ztnegbin", "ztpois"),
```

```

    misclass = NULL,
    latent_classes = 1L,
    control = NULL
)

```

The current implementation focuses on **validation and parsing**. Model fitting (EM algorithm) will be implemented later.

2 Main arguments

Argument	Description
<code>data</code>	One row per observed unit.
<code>captures</code>	One-sided formula defining capture indicators.
<code>capture_formula</code>	Log-linear capture model. <code>.latent</code> denotes the latent class.
<code>outcome</code>	NULL, a single outcome variable, or <code>c(lower=..., upper=...)</code> .
<code>outcome_formula</code>	Mean model for the outcome distribution.
<code>outcome_dist</code>	Currently "ztnegbin" or "ztpois".
<code>misclass</code>	Misclassification specification for categorical variables.
<code>latent_classes</code>	Number of latent classes.
<code>control</code>	Numerical control options (reserved for future use).

3 Current workflow

```

crc_fit()

  match.arg(outcome_dist)
  validate_crc_input()
  parse_captures()
  parse_outcome()
  parse_misclass()
  parse_capture_formula()
  parse_outcome_formula()

```

At this stage the function returns an **unfitted model object** containing parsed specifications.

4 Parser responsibilities

4.1 `parse_captures()`

- validates the capture formula;
- verifies binary capture indicators;
- checks missing values;
- checks that every unit is observed by at least one source;
- returns a `crc_captures` object.

4.2 `parse_outcome()`

Supports:

- `NULL`;
- one exact outcome column;
- interval/right-censored outcomes via `lower/upper`.

Returns a standardized table with:

- `lower`
- `upper`
- `status`

4.3 `parse_capture_formula()`

Validates:

- one-sided formula;
- intercept;
- untransformed variables only;
- presence of all capture main effects;
- hierarchical interactions;
- `.latent`;
- allowed interactions:
 - `capture × capture`,
 - `capture × .latent`.

Returns a `crc_capture_formula` object.

4.4 `parse_outcome_formula()`

Validates:

- one-sided formula;
- intercept;
- categorical predictors;
- hierarchical interactions;
- optional use of `.latent`.

Returns a `crc_outcome_formula` object.

4.5 `parse_misclass()`

Expected structure:

```
misclass = list(  
  occupation = list(  
    matrix = Pi_occ,  
    true_if = ~ source_1  
  ),  
  contract = list(  
    matrix = Pi_contract  
  )  
)
```

Responsibilities:

- validates confusion matrices;
- validates category levels;
- validates `true_if`;
- constructs logical vectors identifying observations with known true category;
- returns a `crc_misclass` object.

5 Returned object

Currently `crc_fit()` returns an object of classes

```
c("crcfit_unfitted", "crcfit")
```

containing:

- parsed capture specification,
- parsed outcome specification,
- parsed capture model,
- parsed outcome model,
- parsed misclassification specification,
- model metadata.

6 Next implementation steps

1. Build internal model matrices.
2. Initialize parameters.
3. Implement the EM algorithm.
4. Compute standard errors.
5. Add `summary()`, `print()`, and `predict()` methods.